Notebook setup

Run the next cell first. In Google Colab it clones the course repository and installs the extra packages, then imports the libraries. If you cloned the repository and run the notebook locally (see the README), it only does the imports.

Bayesian optimization using Gaussian processes

If every function evaluation is expensive, for instance when each evaluation is a bioreactor experiment that takes two weeks, optimizing the process parameters with standard optimization routines (which need hundreds of evaluations) would take forever.

The idea is to approximate the function with a Gaussian process. In other words, the function values are assumed to follow a multivariate Gaussian distribution whose covariance is given by a GP kernel between the parameters. A smart choice of the next parameter to evaluate is then made with an acquisition function computed on the GP posterior, which is much quicker to evaluate than the real function.

Bayesian optimization repeats 4 steps:

1. fit a GP to all measurements so far
2. compute the acquisition function from the GP prediction and its uncertainty
3. run the next experiment where the acquisition function is highest
4. add the new measurement and repeat

Kernels and the multivariate Gaussian distribution behind the GP are explained in `03_GP.ipynb`.

Problem definition

We are interested in solving

$$x^* = \arg\max_x f(x) \tag{1}$$

under the constraints that:

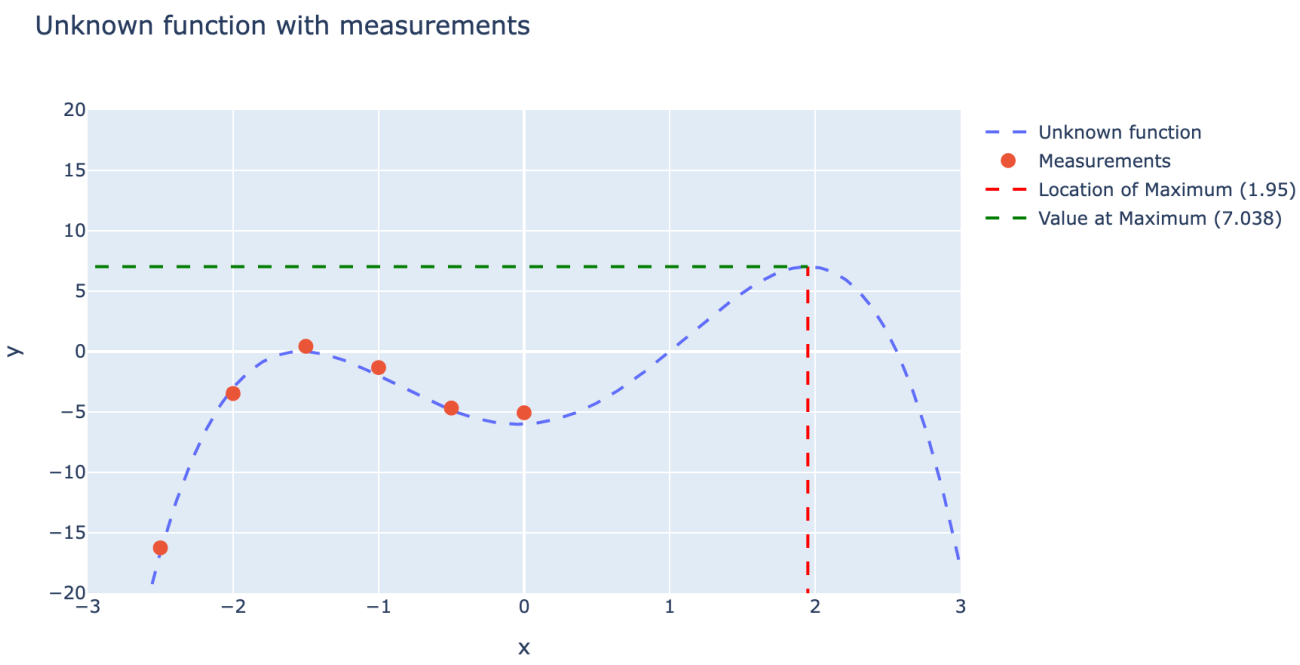
- f is a black box for which no closed form is known (nor its gradients)
- f is expensive to evaluate
- evaluations of $y = f(x)$ may be noisy

Create the function to optimize

Here we first create an originator function that is nonlinear, and we add noise to its observations. A few initial measurements are taken on the left side of the domain.

```
""" Fix seed to control randomness """
np.random.seed(42)
""" Noise in function observations """
EPS = 0.5
""" Define initial experimental points locations """
XPOINTS = np.arange(-2.5,0.5,0.5)
```

```
# Function definition (unknown to the optimizer)
xgrid = np.round(np.arange(-3,3,0.01),4)
def f(x,eps=0.):
    return
-1*np.power(x,4)+0.5*np.power(x,3)+6*np.power(x,2)+0.5*np.power(x,1)-6+eps*np.random.
normal(0,1,x.size)
fgrid = f(xgrid)
# Obtain observation measurements with noise
xpoints = XPOINTS
ypoints = f(xpoints,EPS)
```



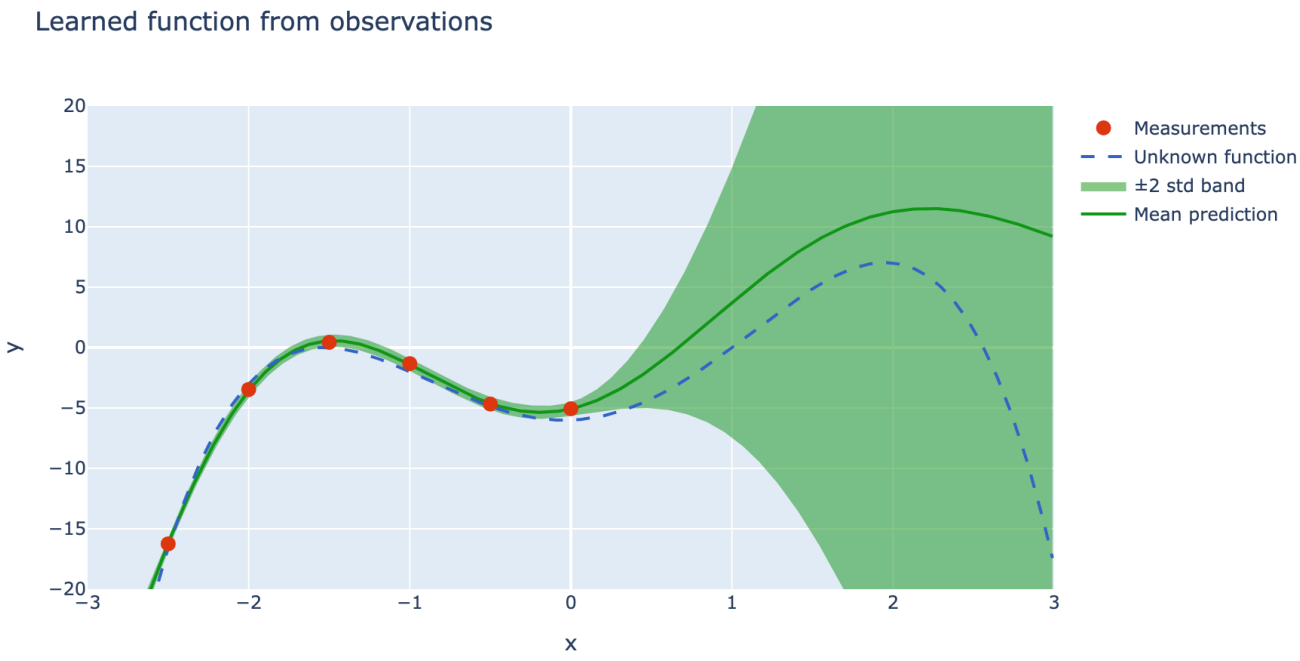
The goal is to find the maximum. If we could sample our function an unlimited number of times, we could just look up the answer. However, we have limited resources, so we can only take a few experimental observations and have to approximate (model) the behaviour in the remaining places.

Fit a Gaussian process model

We use a Gaussian process with a standard RBF kernel to learn the function above from the measurements.

```
# Define Model
kernel = 1.0 * RBF(length_scale=1e-1, length_scale_bounds=(1e-3, 1e3)) +
WhiteKernel(noise_level=1e-2, noise_level_bounds=(1e-2, 1e1))
gpr = GaussianProcessRegressor(kernel=kernel, random_state=0, alpha=0,
n_restarts_optimizer=3).fit(xpoints.reshape(-1,1), ypoints.reshape(-1,1))
print("Fitted kernel:", gpr.kernel_)
# Use model to make predictions
xpred = xgrid
mean, std = gpr.predict(xpred.reshape(-1, 1), return_std=True)
mean, std = mean.ravel(), std.ravel()
ypred = np.vstack([mean - 2*std, mean, mean + 2*std])
```

```
Fitted kernel: 39**2 * RBF(length_scale=1.65) + WhiteKernel(noise_level=0.0388)
```



The ±2 std band is narrow close to the measurements and wide on the right, where there are none. The rising mean there only continues the trend of the last points: it is not knowledge about the function.

Define the acquisition function

The acquisition function represents our objective for choosing the next experiment. It guides us towards the maximum value and its location.

Exploration / exploitation

The simplest acquisition function one can think of is the conditional mean itself. This fully exploits the current model. We can add the uncertainty estimate as an exploration term to also investigate regions with high uncertainty or few observations.

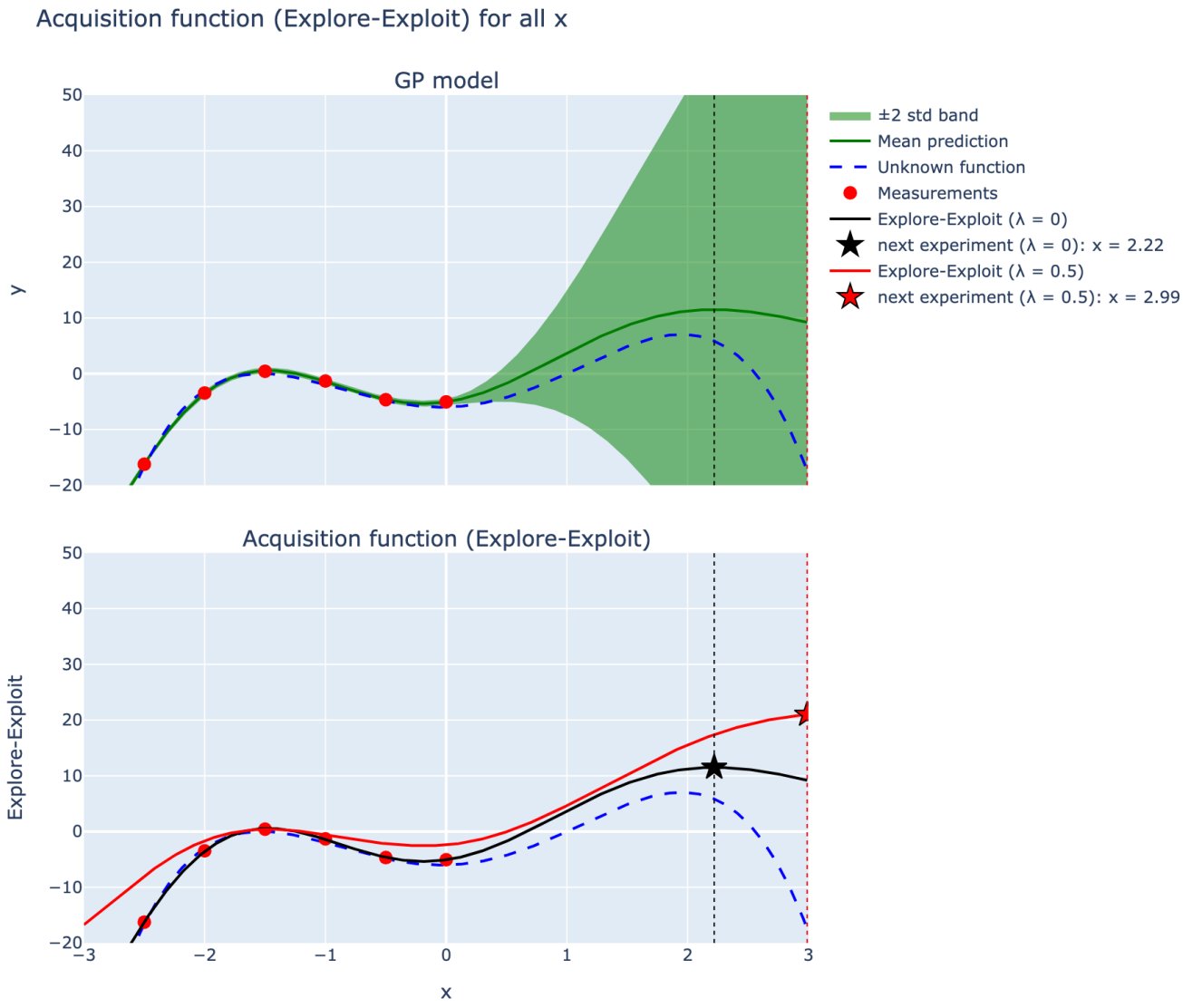
You can define your own acquisition function. The function demonstrated below balances exploration and exploitation with the weight λ (`LAMBDA1` , `LAMBDA2` below). It is a weighted version of the upper confidence bound (UCB):

$$UCB(x) = (1-\lambda)\,\mu_{GP}(x) + \lambda\,\sigma_{GP}(x) \tag{2}$$

Note that the `bayesian-optimization` package used later defines UCB differently, as $\mu_{GP}(x) + \kappa\,\sigma_{GP}(x)$, with its own parameter κ.

```
""" Exploration / Exploitation tradeoff """
LAMBDA1 = 0
LAMBDA2 = 0.5
```

```
# Calculate EE (Explore-Exploit)
eegrid1 = (1-LAMBDA1)*ypred[1,:] + LAMBDA1*((ypred[1,:]-ypred[0,:])/2)
eegrid2 = (1-LAMBDA2)*ypred[1,:] + LAMBDA2*((ypred[1,:]-ypred[0,:])/2)
```



Probability of improvement

Another good acquisition function is the probability of improvement: the probability that the function value at x exceeds the best value observed so far, $f(x^+)$,

$$PI(x) = P\left(f(x) \geq f(x^+)\right) = 1 - \Phi\left(\frac{f(x^+) - \mu_{GP}(x)}{\sigma_{GP}(x)}\right) \tag{3}$$

where Φ is the cumulative distribution function of the standard normal distribution. To demonstrate it, we evaluate the model at a new point and compare the prediction with the maximal value observed so far.

```
""" New evaluation point """
XNEW = 1.5
```

```
# Obtain prediction at the new point
xnew = XNEW
(xnew_mean,xnew_std) = gpr.predict(np.array(xnew).reshape(1,-1),return_std=True)
xnew_mean, xnew_std = xnew_mean.item(), xnew_std.item()
print('Predicted mean and std at x = '+str(xnew))
print("Mean: ",float(xnew_mean))
print("Std: ",float(xnew_std))

# Actual maximum value
y_max = np.max(ypoints)
print("Observed maximum function value so far at y = "+str(round(y_max,4)))
```

```
Predicted mean and std at x = 1.5
Mean:  8.662260233196776
Std:  12.04328594358025
Observed maximum function value so far at y = 0.4348
```

Compute the distribution

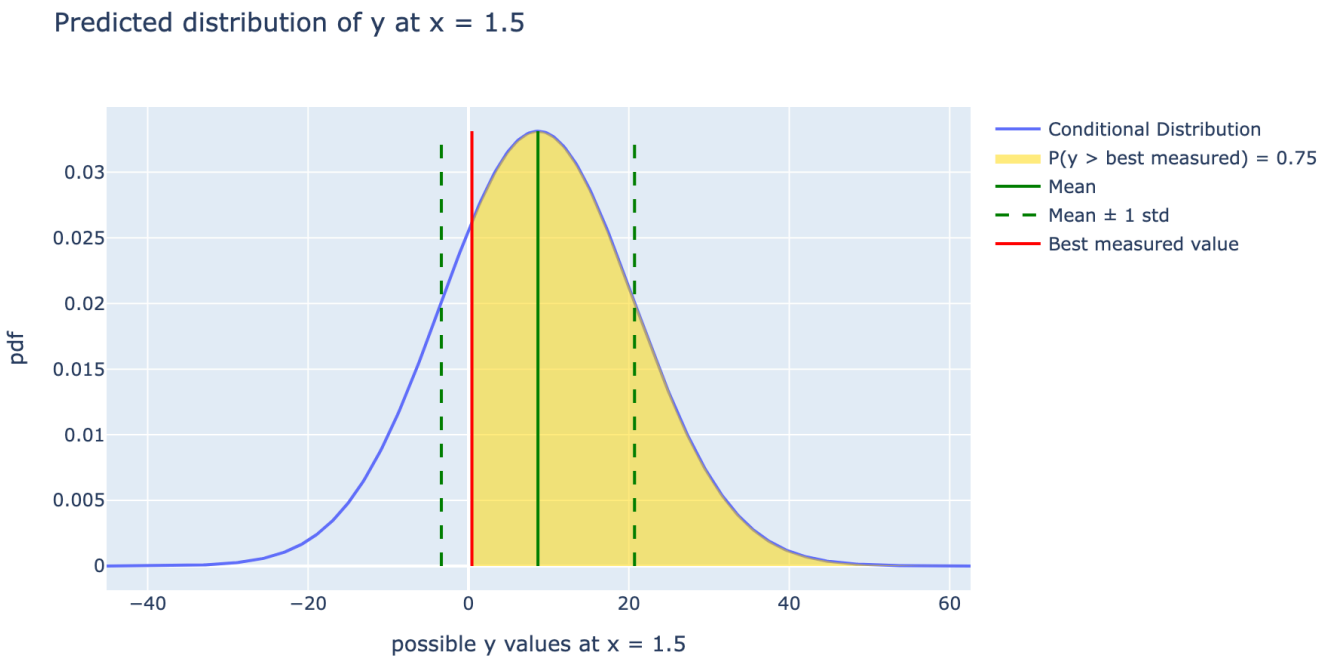
Since the prediction at the new point `XNEW` is normally distributed with mean `xnew_mean` and standard deviation `xnew_std`, we can plot the full distribution at the new point.

This distribution represents the possible observations according to the model. The solid red line is the actually observed maximum value so far, `y_max`.

The probability of improvement is the probability of observing a value from this conditional distribution that is greater than the observed maximum: the shaded area to the right of the red line.

```
# Probability of improvement at the new point
PI = 1-norm(float(xnew_mean), float(xnew_std)).cdf(y_max)
print("Probability of improvement at x = "+str(xnew)+": ", round(PI,4))
# Samples from the conditional distribution (for the plot range)
xdist = np.random.randn(100000)*float(xnew_std) + float(xnew_mean)
hist_data = [xdist]
group_labels = ['Conditional Distribution']
```

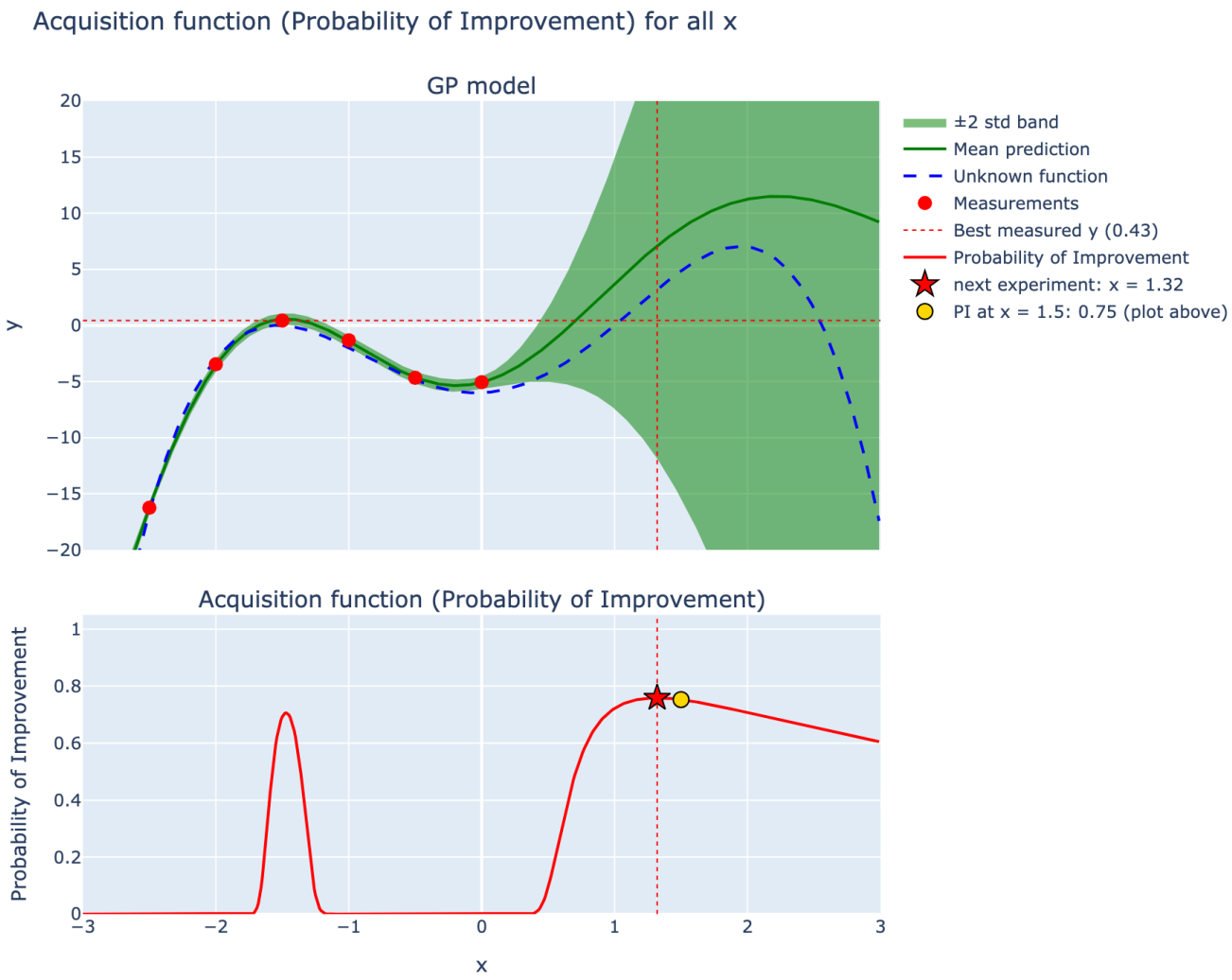
```
Probability of improvement at x = 1.5:  0.7527
```



Compute the acquisition function for all x

Now we compute the probability of improvement for all x values of the grid.

```
# Calculate PI (Probability of Improvement) for all x
mu, sd = ypred[1,:], (ypred[1,:]-ypred[0,:])/2
pigrid = 1 - norm(mu, sd).cdf(y_max)
```



Given the acquisition function, the next point at which we would like to take a measurement is the value of x for which the acquisition function is the highest (the star). The narrow peak near x = –1.5 shows the weakness of PI: an improvement there is likely, but it would be tiny, because the GP is confident that the function stays close to the best measurement.

Expected improvement

PI asks how likely an improvement is; the expected improvement (EI) asks how large the improvement is expected to be. Only improvements count, so possible values below the best observation contribute zero:

$$EI(x) = E\left[\max\left(f(x) - f(x^+) - \xi,\, 0\right)\right] \tag{4}$$

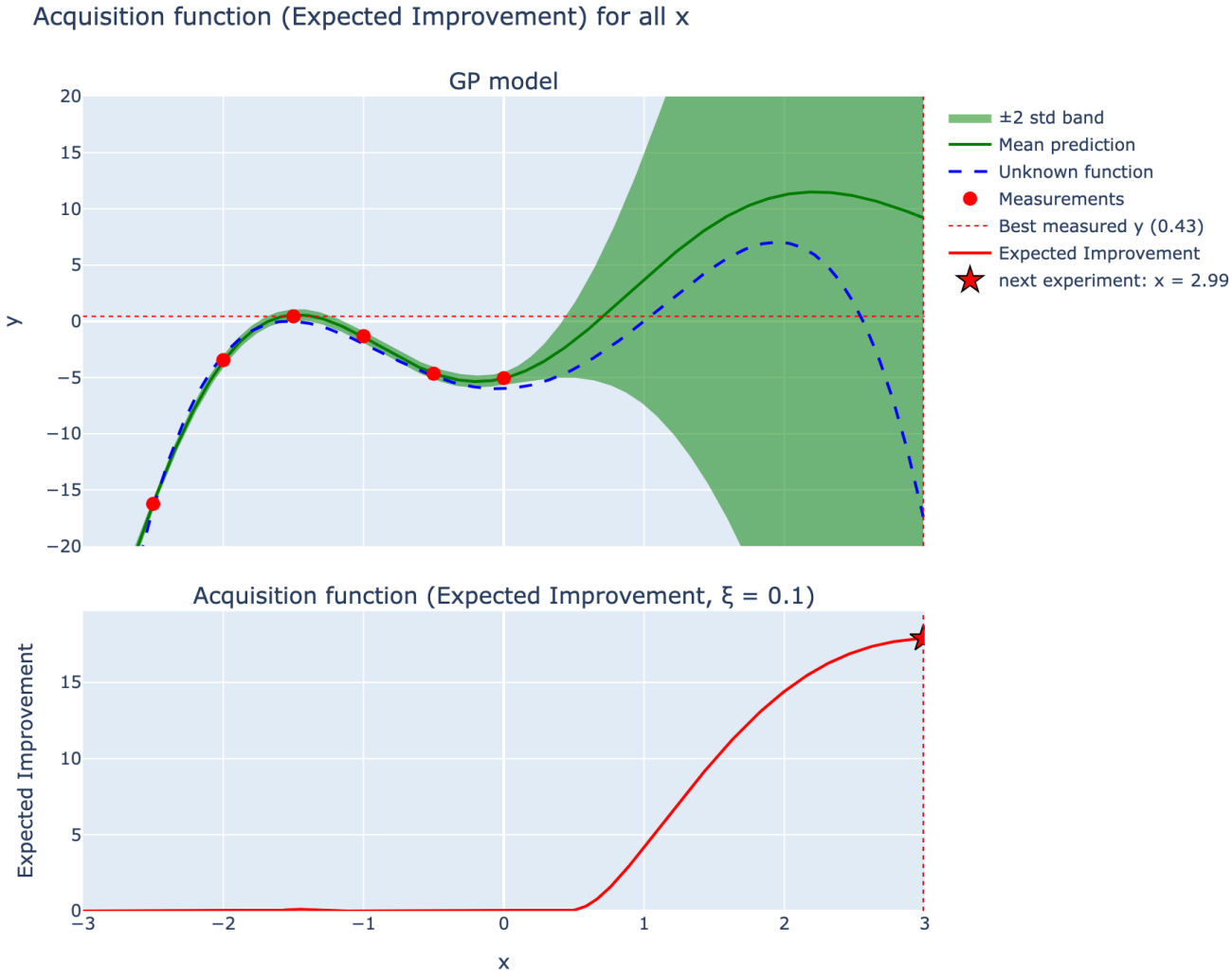
For the Gaussian prediction of the GP, it has the closed form

$$EI(x) = \left(\mu_{GP}(x) - f(x^+) - \xi\right)\Phi(z) + \sigma_{GP}(x)\,\phi(z), \qquad z = \frac{\mu_{GP}(x) - f(x^+) - \xi}{\sigma_{GP}(x)} \tag{5}$$

where Φ and ϕ are the cumulative distribution function and the density of the standard normal distribution. The first term is large where the mean is above the best observation (exploitation), the

second where the uncertainty is large (exploration). The parameter ξ = `xi` means: ignore improvements smaller than ξ.

```
# Calculate EI (Expected Improvement) with the closed-form expression for a Gaussian posterior
xi = 0.1
z = (mu - y_max - xi) / sd
eigrid = (mu - y_max - xi)*norm.cdf(z) + sd*norm.pdf(z)
```



Compared with PI, the bump near x = −1.5 has almost disappeared: EI takes into account that the improvement there would be tiny.

Run Bayesian optimization

Now that we have defined the optimization problem, we run the Bayesian optimization with the `bayesian-optimization` package and see whether we are able to iteratively find the maximum.

From here on:

- `bayes_opt` runs the loop: fit the GP, maximize the acquisition function, run the next experiment

- the observations are noisy, with noise level `EPS`
- the first run uses the package's default acquisition function, the upper confidence bound $UCB(x) = \mu_{GP}(x) + \kappa\,\sigma_{GP}(x)$ with $\kappa = 2.576$; the step-by-step run uses `acq_fn`, the same UCB with κ = `KAPPA`
- the GP uses a Matérn kernel

Watch how many experiments it takes to get close to the maximum.

Target function

Let's define a 1-D target function with multiple local maxima to test and visualize how the `BayesianOptimization` package works. Three target functions are prepared in the code (the second and third ones are commented out):

$$f_1(x) = -x^4 + 0.5x^3 + 6x^2 + 0.5x - 6 + \epsilon \tag{6}$$

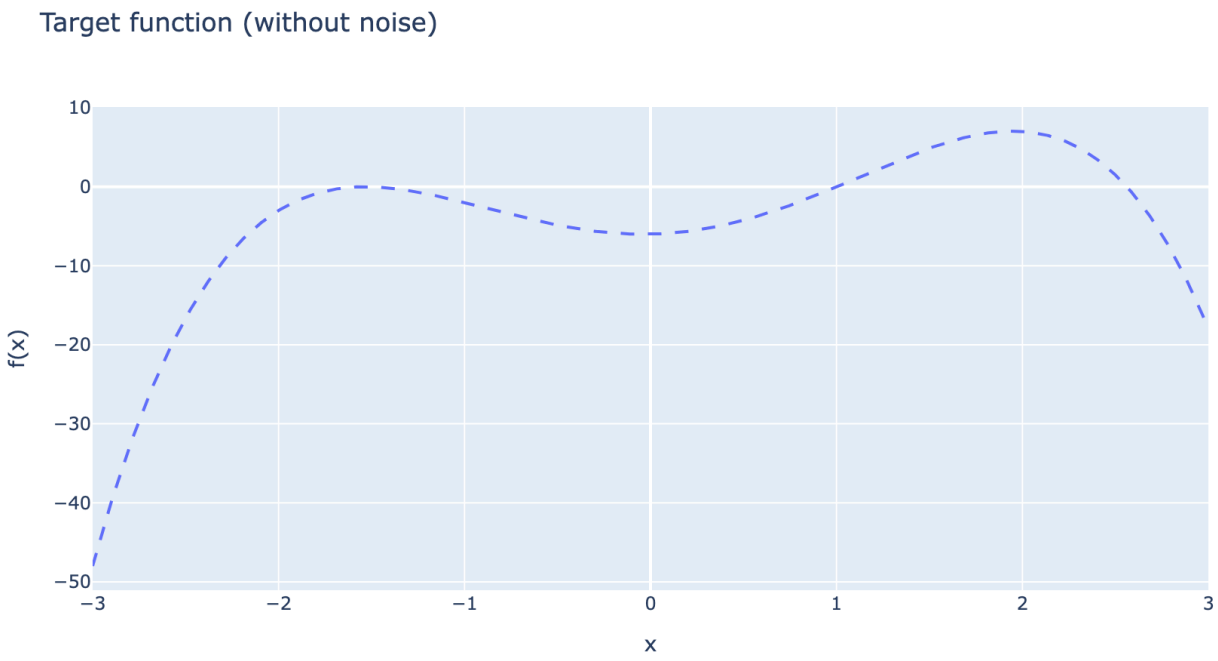
$$f_2(x) = e^{-(x-2)^2} + e^{-0.1(x-6)^2} + 1/(x^2+1) \tag{7}$$

$$f_3(x) = 0.5\sin(x) - 0.02(10-x)^2 + 2 \tag{8}$$

Function	Domain	True maximum	What makes it hard
f_1 polynomial	−3 to 3	7.04 at x = 1.95	a local peak near −1.5
f_2 exponential	−2 to 10	1.40 at x = 2.0	a sharp peak next to a broad hill
f_3 sin	0 to 20	2.41 at x = 8.0	many small peaks on a trend

Notice that in practice this function is unknown; the only information we have is obtained by sequentially probing it at different points. Bayesian optimization works by constructing a posterior distribution of functions that best fits the observed data and choosing the next probing point by balancing exploration and exploitation.

```
""" Noise level """
EPS = 0.5
""" Function to optimize """
def f(x,eps=EPS):
    x = np.array(x)
    return -1*(x**4)+0.5*(x**3)+6*(x**2)+0.5*x-6 + eps*np.random.normal(0,1,x.shape)
# domain (-3,3)
    #return np.exp(-(x - 2)**2) + np.exp(-(x - 6)**2/10) + 1/ (x**2 + 1)   # domain
(-2,10)
    #return np.sin(x)/2 - ((10 - x)**2)/50 + 2 # use domain (0,20)
""" Domain / range of values"""
domain = (-3,3)
#domain = (-2,10)
#domain = (0,20)
```



We pass the target function to be maximized, its variable and the corresponding range to the optimizer. A minimum of 2 initial guesses is necessary to kick-start the algorithm; these are chosen at random here. The optimizer then runs 30 iterations, each time fitting the GP and probing the point with the highest acquisition value.

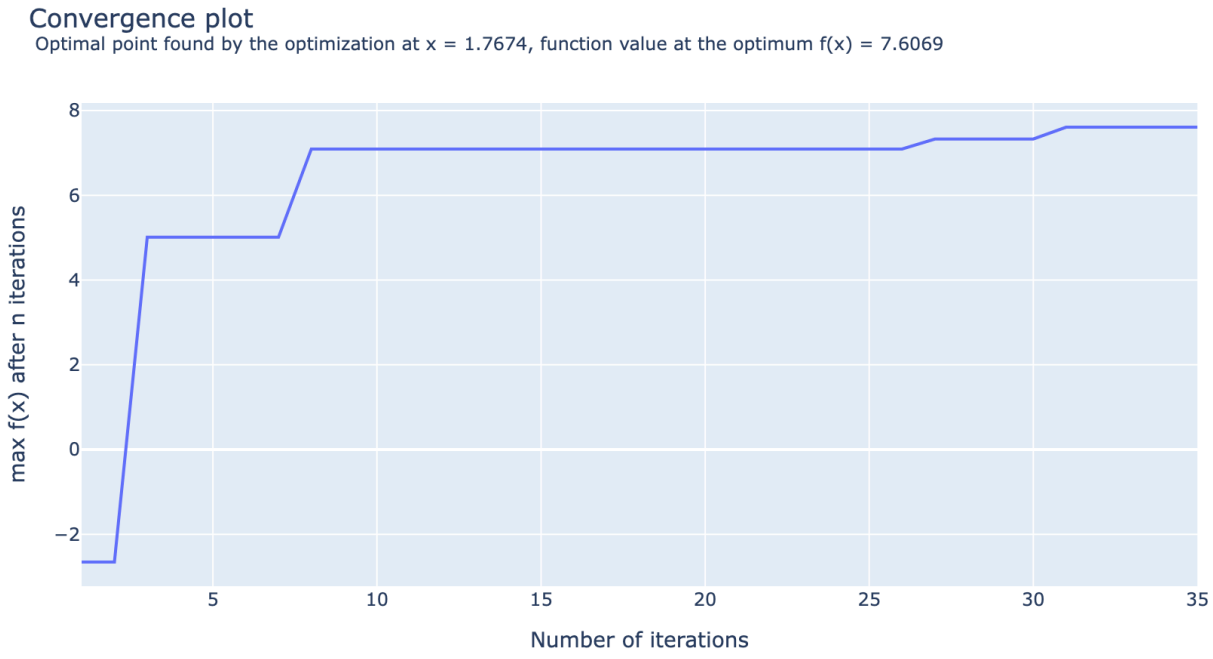
```
# Define optimizer (verbose=2 prints every probed point)
optimizer = BayesianOptimization(f, {'x': domain}, random_state=42, verbose=2)
# alpha is the assumed noise variance of the observations
optimizer._gp.set_params(**dict(alpha=1))
# Run the optimization: 5 random initial points followed by 30 Bayesian iterations
# No acquisition_function is passed, so bayes_opt uses its default: UCB with
kappa=2.576
optimizer.maximize(init_points=5, n_iter=30)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 1         | -2.653    | -0.7528   |
| 2         | -4.248    | 2.704     |
| 3         | 5.012     | 1.392     |
| 4         | -3.705    | 0.592     |
| 5         | -4.071    | -2.064    |
| 6         | 3.789     | 1.365     |
| 7         | 4.396     | 1.594     |
| 8         | 7.091     | 1.911     |
| 9         | -48.5     | -3.0      |
| 10        | -17.64    | 3.0       |
| 11        | -6.298    | -0.0273   |
| 12        | -0.9223   | 0.9395    |
| 13        | -1.294    | -1.11     |
| 14        | 6.698     | 1.948     |
| 15        | 6.672     | 1.94      |
| 16        | 7.001     | 1.919     |
| 17        | 4.571     | 1.386     |
| 18        | 6.66      | 2.085     |
```

```
| 19        | 0.8745    | -1.369    |
| 20        | 6.649     | 1.658     |
| 21        | 6.335     | 2.134     |
| 22        | 5.685     | 1.579     |
| 23        | -5.043    | -0.4214   |
| 24        | 6.676     | 2.091     |
| 25        | 6.065     | 1.638     |
| 26        | 1.112     | 1.158     |
| 27        | 7.326     | 2.099     |
| 28        | 6.796     | 1.8       |
| 29        | 6.839     | 2.159     |
| 30        | 6.314     | 1.753     |
| 31        | 7.607     | 1.767     |
| 32        | 6.067     | 2.182     |
| 33        | 6.059     | 1.73      |
| 34        | 5.632     | 1.777     |
| 35        | 6.256     | 2.183     |
=====================================
```

Convergence plot

The best function value found so far is plotted against the number of iterations.



Acquisition function progression

To understand what happens in every iteration, we now run the optimization step by step and plot the GP posterior (top) together with the acquisition function (bottom). The same acquisition function `acq_fn` (upper confidence bound with `kappa=KAPPA`) is used both for the plot and by the optimizer to choose the next point, so the star marks the point that is probed in the next step.

Define the optimization that we investigate sequentially. It starts with 2 random points and uses a Matern kernel in the GP.

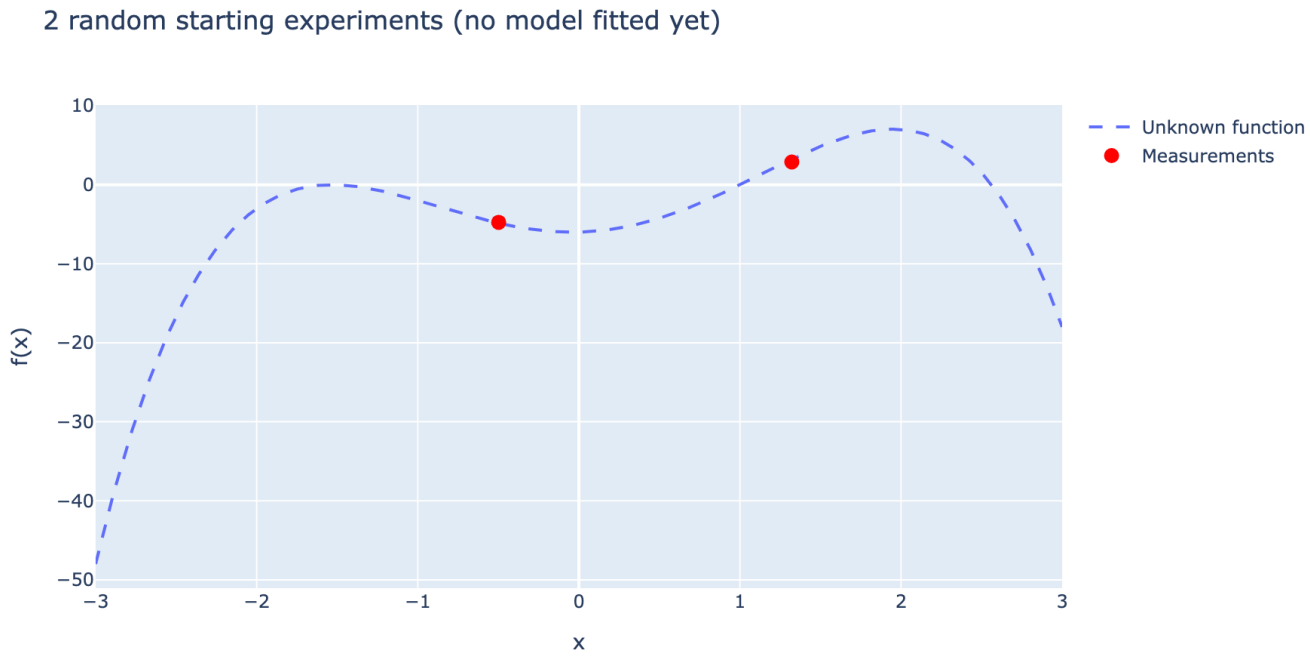
```
# Grid for plotting
x = xgrid.reshape(-1, 1)
y = f(xgrid,eps=0)
# Define the optimization and probe 2 random initial points
N_INIT = 2
optimizer = BayesianOptimization(f, {'x': domain}, random_state=1, verbose=2)
optimizer.maximize(init_points=N_INIT, n_iter=0)
# Modify the GP model: Matern kernel and observation noise
kernel = Matern(length_scale=1, nu=2.5, length_scale_bounds=(0.5,1e3))
optimizer._gp.set_params(**dict(alpha=0.1, kernel=kernel))
print("GP kernel:", optimizer._gp.kernel)
# Acquisition function used both for plotting and for choosing the next point
KAPPA = 5
acq_fn = UtilityFunction(kind="ucb", kappa=KAPPA, xi=0)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 1         | -4.751    | -0.4979   |
| 2         | 2.882     | 1.322     |
=====================================
GP kernel: Matern(length_scale=1, nu=2.5)
```

Two random points

The optimization starts with 2 points that are chosen at random, before any model is fitted. From the next step on, a GP is fitted to all measurements and the acquisition function chooses the next experiment.

The lower bound 0.5 on the length scale of the Matern kernel is a prior assumption: the function does not vary faster than that. It is needed because 2 points cannot determine the length scale.

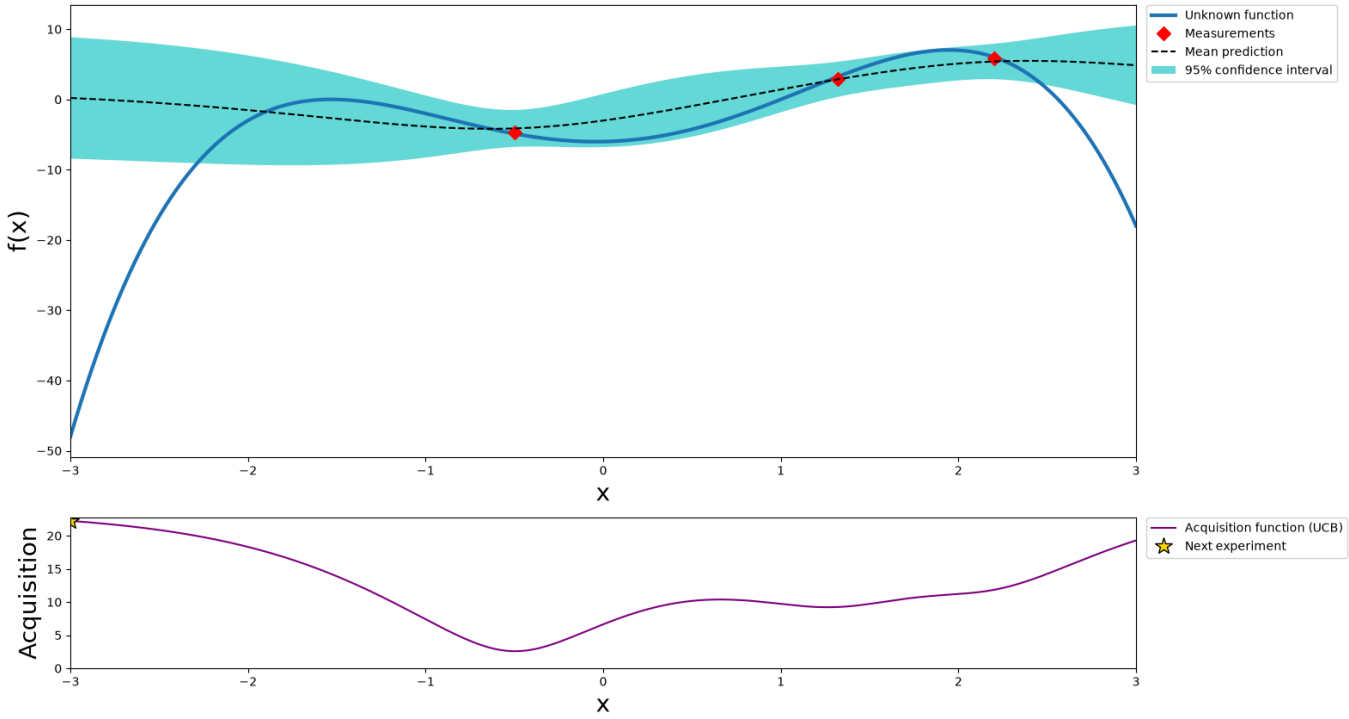


After one step of Bayesian optimization

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y, acq_fn, domain)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 3         | 5.897     | 2.201     |
=====================================
```

Gaussian process and acquisition function after 1 BO iterations

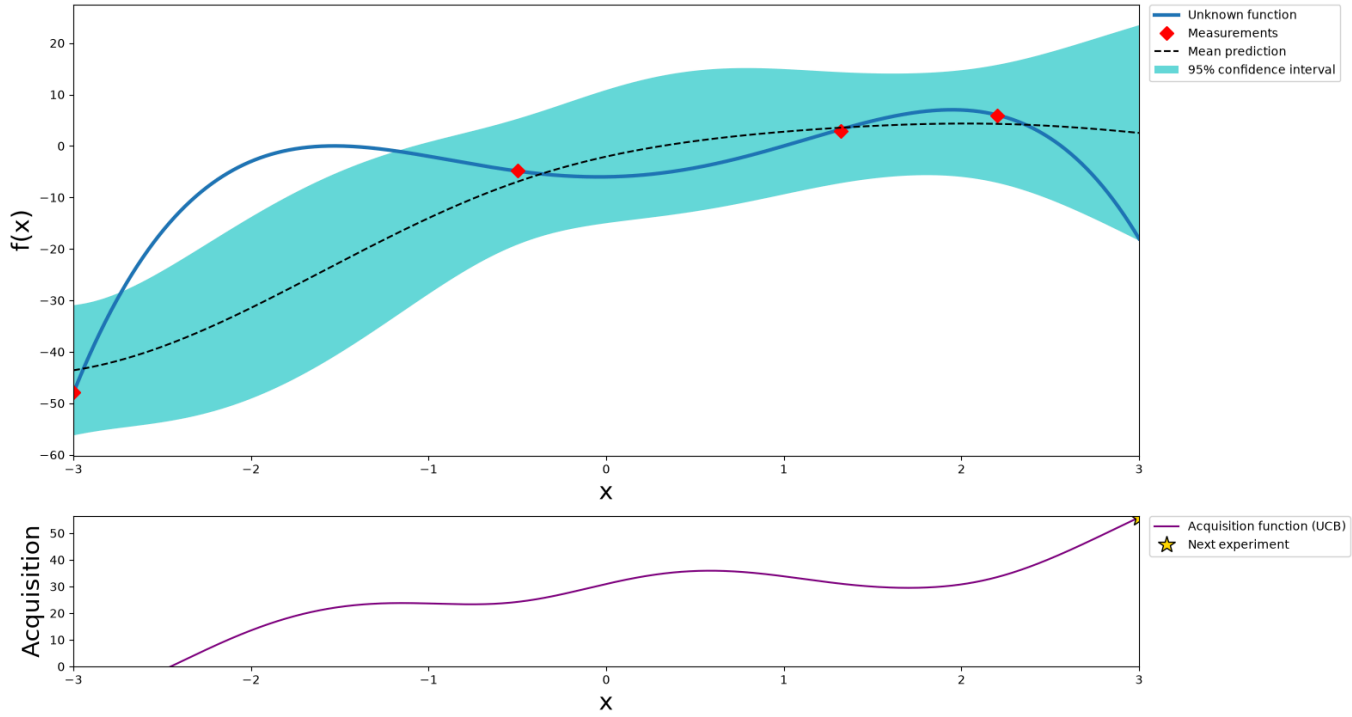


After two steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y, acq_fn, domain)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 4         | -47.9     | -3.0      |
=====================================
```

Gaussian process and acquisition function after 2 BO iterations

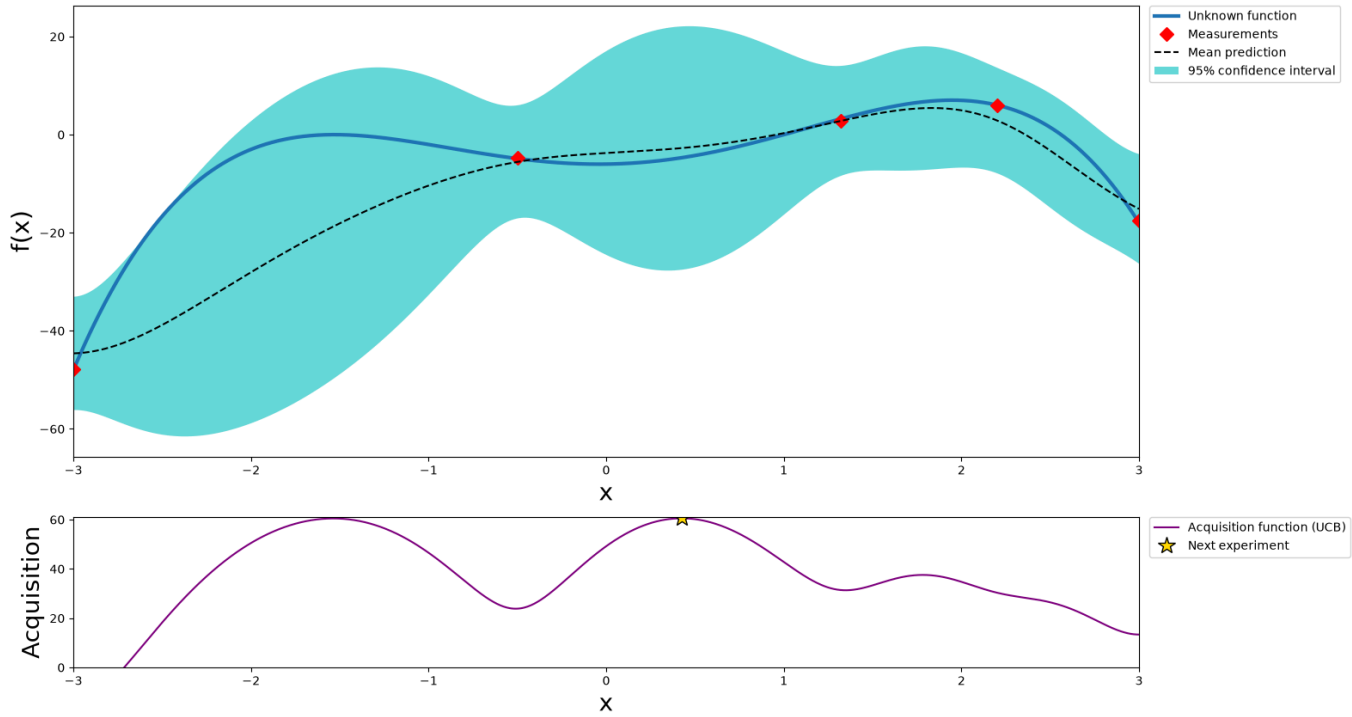


After three steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y, acq_fn, domain)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 5         | -17.48    | 3.0       |
=====================================
```

Gaussian process and acquisition function after 3 BO iterations

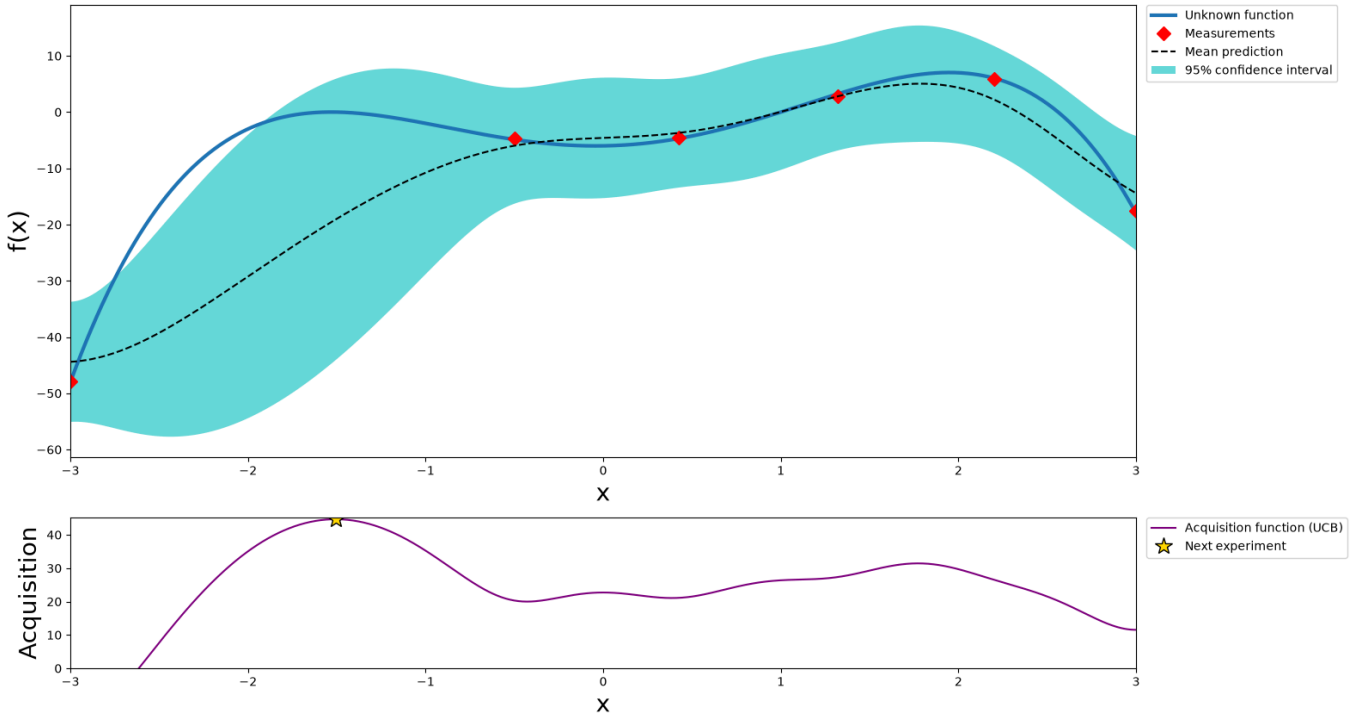


After four steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y, acq_fn, domain)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 6         | -4.47     | 0.4249    |
=====================================
```

Gaussian process and acquisition function after 4 BO iterations

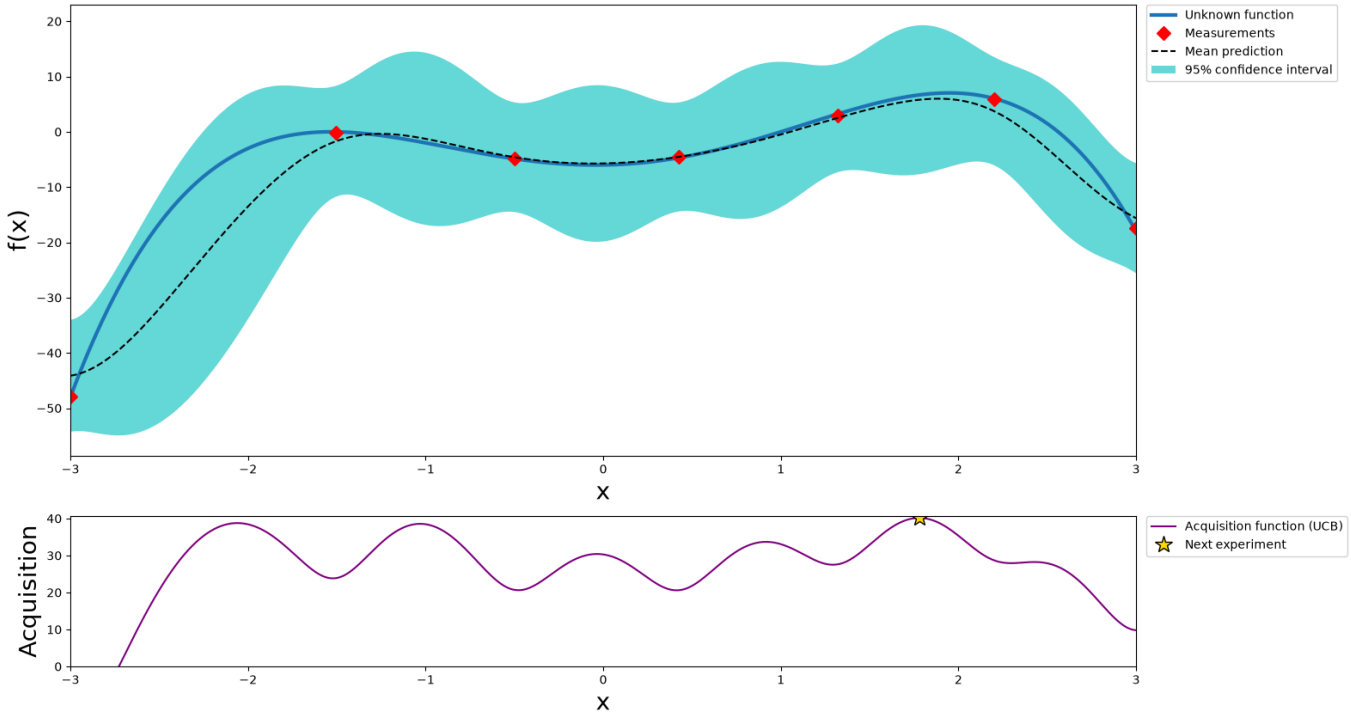


After five steps

```
optimizer.maximize(init_points=0, n_iter=1, acquisition_function=acq_fn)
plot_gp(optimizer, x, y, acq_fn, domain)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 7         | -0.1956   | -1.507    |
=====================================
```

Gaussian process and acquisition function after 5 BO iterations



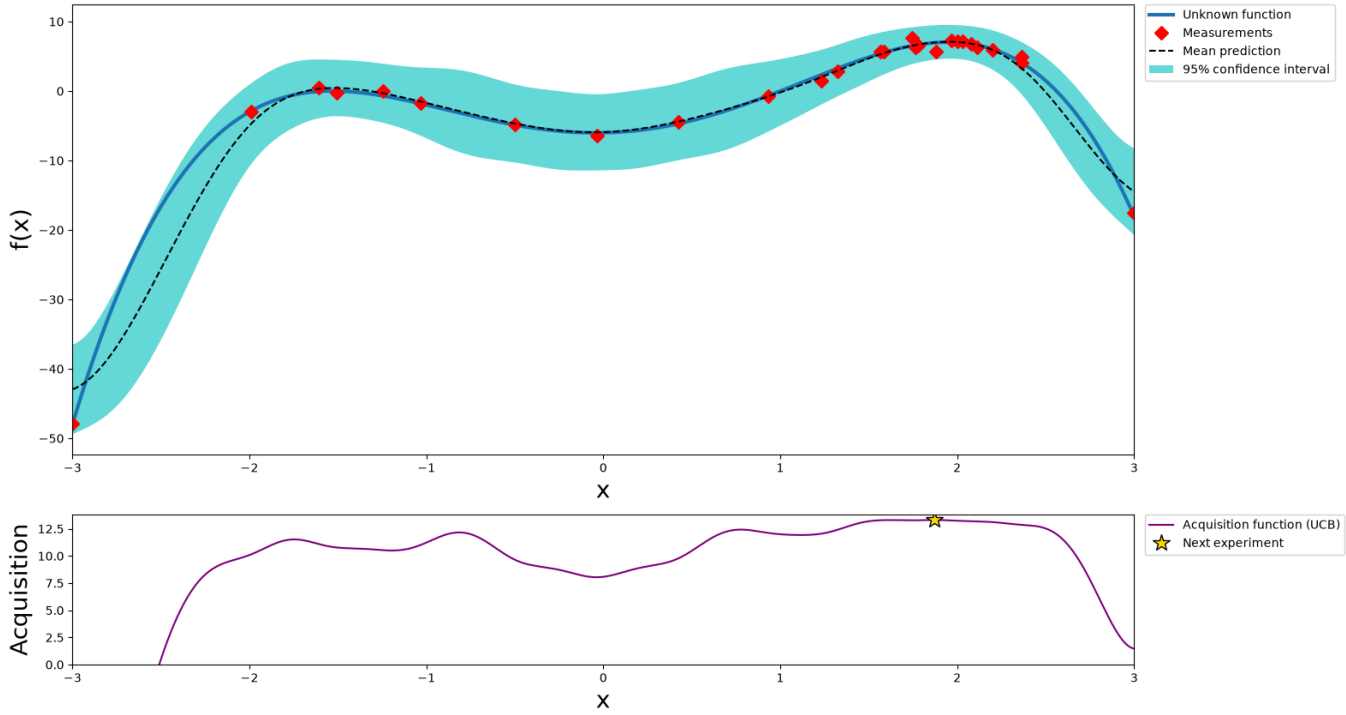
After 25 steps

After 20 more iterations the optimizer has concentrated most experiments (about two thirds) near the maximum, so the GP describes the function well there. It probes regions far from the optimum only occasionally (here mostly around the second local maximum near x = -1.5), so the GP remains more uncertain there, especially towards the edges of the domain.

```
optimizer.maximize(init_points=0, n_iter=20, acquisition_function=acq_fn)
plot_gp(optimizer, x, y, acq_fn, domain)
```

```
|   iter    |  target   |     x     |
-------------------------------------
| 8         | 6.517     | 1.779     |
| 9         | -1.735    | -1.034    |
| 10        | -2.991    | -1.991    |
| 11        | -0.7005   | 0.93      |
| 12        | 5.651     | 1.88      |
| 13        | -6.447    | -0.03451  |
| 14        | 4.926     | 2.366     |
| 15        | 5.628     | 1.567     |
| 16        | 7.205     | 2.029     |
| 17        | 7.221     | 2.003     |
| 18        | 5.744     | 1.588     |
| 19        | 4.122     | 2.366     |
| 20        | 0.541     | -1.606    |
| 21        | 7.289     | 1.968     |
| 22        | 1.513     | 1.234     |
| 23        | 6.27      | 1.765     |
| 24        | 6.338     | 2.112     |
| 25        | 7.717     | 1.747     |
| 26        | 6.838     | 2.08      |
| 27        | -0.002049 | -1.246    |
=====================================
```

Gaussian process and acquisition function after 25 BO iterations




```
# Summary of the step-by-step optimization (measurements are noisy, so the best
measured y can lie above the true maximum)
best = optimizer.max
first = next(k+1 for k, r in enumerate(optimizer.res) if r["target"] >=
best["target"] - 0.1)
print(f"Best measured y = {best['target']:.2f} at x = {best['params']['x']:.2f}
({len(optimizer.res)} experiments, including the {N_INIT} random ones)")
print(f"First experiment within 0.1 of this best: experiment {first}")
# True maximum of the target function (fine grid, without noise)
xfine = np.linspace(domain[0], domain[1], 100001)
yfine = f(xfine, eps=0)
print(f"True maximum: y = {yfine.max():.2f} at x = {xfine[np.argmax(yfine)]:.2f}")
```

```
Best measured y = 7.72 at x = 1.75 (27 experiments, including the 2 random ones)
First experiment within 0.1 of this best: experiment 25
True maximum: y = 7.04 at x = 1.95
```

Wrap up

- A local design only finds a local optimum.
- A Gaussian process gives, for every x, both a prediction and an uncertainty.
- Acquisition functions balance exploitation (high predicted mean) and exploration (high uncertainty).
- Bayesian optimization finds the optimum with few, well-chosen experiments.

Tasks

1. Switch to the second target function f_2 (uncomment it in the function definition and set `domain = (-2,10)`). Does the optimizer find the global maximum or does it get stuck in a local one?

2. Change `KAPPA` (defined where the step-by-step optimizer is created) to 1 (exploitation) and to 10 (exploration) and repeat the step-by-step optimization. How does the sequence of probed points change?
3. Increase the noise `EPS` to 2. How many more iterations does the optimizer need to find the maximum?

```
# Your code here
```